

ДОМАШНЕЕ ЗАДАНИЕ 2: ФИЛЬТР ЗАМЕТОК

Сейчас приложение умеет выводить список заметок и помечать их как важные. Нужно добавить кнопку фильтрации списка. Фильтрация должна выполняться на стороне сервера. Обратите внимание на особенность взаимодействия фильтрации с другими запросами.

Ссылки на исходники проектов для ДЗ для групп (все группы имеют доступ к файлам любой из них):

ПД-412	https://drive.google.com/file/d/1q9VWHLJfe8kfZ1ACGjRdyxDwGJyQeaYV/view?usp=sharing
П-318	https://drive.google.com/file/d/1DJhRkBABZi37aVLOkGvviM-36xLVPfvt/view?usp=sharing
П-319	https://drive.google.com/file/d/1-QwEyl4ApIGQaNO0Uwwwv7jh2KOXSgLUw/view?usp=sharing

1.1 Ход работы

1.1.1 Добавить форму с <select> для фильтрации в Index Page

Внутри разметки заметки нужно добавить маленькую форму с методом GET и с <select name="filter">, который передаст параметр в метод GetAll. Метод GetAll должен обзавестись необязательным параметром filter типа ImportanceFilter.

```
public List<Note> GetAll(ImportanceFilter filter);
```

Необходимо завести enum ImportanceFilter, который имеет 3 состояния:

```
public enum ImportanceFilter {  
    All,  
    ImportantOnly,  
    NotImportantOnly  
}
```

Поведение каждого из них вытекает из самого названия состояния. All – нет фильтрации, ImportantOnly – вернет только важные заметки, NotImportantOnly – вернет только неважные заметки.

1.1.2 Добавить обработку фильтрации в сервисы

В принципе достаточно реализовать обработку фильтра в лоб – просто в самой функции Where решать, какой элемент брать, а какой нет через switch-case по filter.

```
switch (filter)
{
    case ImportanceFilter.ImportantOnly:
    case ImportanceFilter.NotImportantOnly:
    default:
}
```

Для InMemoryNoteService такой подход полностью оправдан – там никогда не будет достаточно большого количества данных, а вот для SqliteNoteService это приведет к формированию избыточного сложного SELECT-запроса. Поэтому лучшим подходом будет объявить переменную

```
var query = _context.Notes.AsQueryable()
```

И дальше в ветках switch-case присваивать

```
query = query.Where(...)
```

Получится что-то вроде:

```
var query = _context.Notes.AsQueryable();
switch (filter)
{
    case ImportanceFilter.ImportantOnly:
        query = query.Where(...);
        break;
    case ImportanceFilter.NotImportantOnly:
        query = query.Where(...);
        break;
}

return [.. query.OrderBy(entity => entity.CreatedAt)
        .Select(entity => entity.ToNote())
];
```

Такой подход позволяет создавать различные запросы в зависимости от пришедших данных. Если фильтр позволяет отдавать все элементы, то блока Where вообще не будет.

1.1.3 Необходимо обработать фильтрацию во всех остальных запросах

Сейчас, если применить фильтр, то заметки фильтруются, как надо, однако, стоит отправить любой запрос, который отправляется через форму,

как фильтр тут же слетает. Необходимо как-то передать во все остальные запросы параметр для фильтрации. Первым делом, добавляется параметр `filter` во все методы-обработчики запросов, включая `OnFail`. Все эти методы внутри себя будут прокидывать параметр `filter` в `OnFail` и в `RedirectToPage`:

```
Notes = _notes.GetAll(filter);  
...  
RedirectToPage(new { filter } );
```

Последний будет триггерить вызов `OnGet(filter)`, что приведет к получению отфильтрованных данных после выполнения любого запроса.

1.1.4 Добавить `hidden input` во все формы

Однако после обновления всех обработчиков ничего не поменялось, ведь параметр `e` может попасть в них магическим образом. Значит, надо его как-то туда передать. Поэтому в каждую форму, кроме той, которая отвечает за фильтр надо добавить `<input type="hidden" name="filter">`. В качестве `value` он будет принимать `Model.Filter`. Соответственно, в модель необходимо добавить публичное свойство `Filter`, которое будет инициализироваться внутри `OnGet` значением `filter` из параметра. Тогда все остальные формы тоже смогут отправлять значение `filter`.

1.2 Подводные камни

В текущей реализации фильтрация не может корректно работать вместе с перестановкой элементов. Если фильтр применен, то при перестановке элементов, их порядок может прийти в не консистентное состояние из-за того, что длина отфильтрованного массива не соответствует длине исходного, соответственно, переставляться будет только часть элементов. Для выполнения домашнего задания это не критично, однако при желании вы можете придумать способ, как это обойти. Самый простой вариант – заблокировать перемещение, если включен фильтр. То есть, `draggable="false"`, и дополнительно проверка на бэкенде, что если `filter != All`, то `return`. Чуть сложнее – распределять `SortingOrder` только между отфильтрованными элементами на основе их исходных значений. То есть, не присваивать им

значения на основе индексов, а поменять местами их предыдущие значения на основе новых позиций.

1.3 Требования к результату

После выполнения задания:

- 1) На странице есть форма фильтрации с выбором режима отображения заметок.
- 2) Доступны три режима фильтрации: все заметки, только важные, только неважные.
- 3) Фильтрация выполняется на стороне сервера через `INoteService.GetAll(...)`.
- 4) После выбора фильтра на странице отображаются только подходящие заметки.
- 5) При добавлении, удалении или изменении важности заметки выбранный фильтр не сбрасывается.
- 6) В `InMemoryNoteService` и `SqliteNoteService` реализована обработка параметра фильтра
- 7) В `SqliteNoteService` фильтрация строится через `IQueryable` и добавление `Where` только при необходимости

1.4 Вопросы для самопроверки

Вопросы предназначены исключительно для самоконтроля, присылать их вместе с домашним заданием не нужно

- 1) Почему фильтрацию лучше выполнять через GET, а не через POST?
- 2) Зачем сохранять текущий фильтр в модели страницы?
- 3) Почему формы добавления, удаления и изменения важности не получают фильтр автоматически?
- 4) Зачем передавать `filter` через скрытый `input` в остальных формах?
- 5) Почему после POST нужно делать `RedirectToPage(new { filter })`?

6) Чем отличается фильтрация через `List.Where(...)` от построения запроса через `IQueryable` в `SqliteNoteService`?